

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

CYE 5103 : Linux OS and Scripting – Scripting

Reg. Number	Name	Branch
261100690009	Anshul A Salian	CYS

05/11/2026

CYBERSECURITY LOG ANALYSIS USING LINUX TOOLS

Selected Cybersecurity Scenario

Scenario: Detection of SSH Brute-Force Login Attempts
An SSH brute-force attack is a type of cyberattack in which an adversary repeatedly attempts to gain unauthorized access to a system by systematically guessing passwords. Attackers often test multiple username and password combinations in rapid succession.

Linux authentication logs record both successful and failed login activities. Analyzing these system logs helps security analysts identify repeated authentication failures and pinpoint suspicious activity.

Security Significance:

- **Detects unauthorized access attempts** early before privileges can be escalated.
- **Identifies source IP addresses** responsible for high volumes of failed login events.
- **Highlights targeted user accounts** that require immediate hardening.
- **Informs defensive engineering** to implement strict preventative measures like fail2ban, rate-limiting, or key-based authentication.

2. Log File Acquisition

A controlled, simulated authentication log file was generated for this study to replicate typical SSH access attempts on a Linux host, capturing a mix of failed credentials and successful logins.

Environment and Tools Used:

- **Operating System:** Kali Linux
- **Target Log File:** auth_security.log (customized log archive)
- **Core Utilities:** Bash, grep, sed, awk, sort, and uniq

Procedure:

1. Established a dedicated analysis workspace directory in Kali Linux.
2. Created and populated a sample authentication log file named auth_security.log.
3. Injected representative SSH events comprising valid and invalid credentials.

```
┌──(kali㉿kali)-[~/log]
└─$ cat auth_security.log
Sep 07 14:20:11 anshul-kali sshd[2101]: Failed password for invalid user admin from 10.0.2.45 port 51234 ssh2
Sep 07 14:20:15 anshul-kali sshd[2102]: Failed password for invalid user root from 10.0.2.45 port 51235 ssh2
Sep 07 14:20:19 anshul-kali sshd[2103]: Failed password for invalid user test from 10.0.2.45 port 51236 ssh2
Sep 07 14:20:23 anshul-kali sshd[2104]: Failed password for invalid user guest from 10.0.2.45 port 51237 ssh2
Sep 07 14:20:28 anshul-kali sshd[2105]: Failed password for invalid user admin from 10.0.2.45 port 51238 ssh2
Sep 07 14:21:01 anshul-kali sshd[2107]: Failed password for anshul from 10.0.2.18 port 51239 ssh2
Sep 07 14:21:06 anshul-kali sshd[2108]: Failed password for anshul from 10.0.2.18 port 51240 ssh2
Sep 07 14:22:15 anshul-kali sshd[2110]: Accepted password for anshul from 10.0.2.5 port 51241 ssh2
Sep 07 14:23:02 anshul-kali sshd[2112]: Failed password for invalid user guest from 10.0.2.45 port 51242 ssh2
```

4. Processed and evaluated the records using native Linux command-line filters and a custom Bash automation script

3. Structure of the Log File

Linux authentication logs document system security events in a standardized format.

Key Fields in Each Entry:

- **Timestamp:** Exact date and time when the event occurred.
- **Hostname:** The designated name of the machine generating the event.
- **Process Name:** The specific system daemon responsible (e.g., sshd).
- **Process ID (PID):** The unique identifier assigned to the active process instance.
- **Event Message:** Detailed description of the authentication outcome.
- **Username:** The account targeted in the login attempt.
- **IP Address:** The origin network address of the connection request.

Sample Log Entry Breakdown:

```
Sep 07 14:20:11 anshul-kali sshd[2101]: Failed password for invalid user admin from 10.0.2.45 port 51234 ssh2
```

This specific entry documents an unsuccessful SSH login targeting the non-existent user admin originating from source IP 10.0.2.45.

4. Cybersecurity Investigation Questions

To guide this log analysis exercise, the following core security questions were defined:

1. Which external IP addresses generated failed SSH login attempts?
2. Which IP addresses breached the threshold of three or more failed authentication attempts?
3. How many successful login operations were successfully completed during the timeframe?
4. Were multiple unique usernames targeted concurrently from a single IP address?
5. Which source addresses necessitate deeper threat hunting and incident response procedures?

5. Linux Commands Used for Log Analysis

5.1 Power Filters and Pipelines

Command-line pipelines allow administrators to filter, stream, and transform raw text data efficiently.

1. Isolating Failed Logins:

```
grep "Failed password" auth_security.log
```

```
┌──(kali㉿kali)-[~/log]
└─$ grep "Failed password" auth_security.log
Sep 07 14:20:11 anshul-kali sshd[2101]: Failed password for invalid user admin from 10.0.2.45 port 51234 ssh2
Sep 07 14:20:15 anshul-kali sshd[2102]: Failed password for invalid user root from 10.0.2.45 port 51235 ssh2
Sep 07 14:20:19 anshul-kali sshd[2103]: Failed password for invalid user test from 10.0.2.45 port 51236 ssh2
Sep 07 14:20:23 anshul-kali sshd[2104]: Failed password for invalid user guest from 10.0.2.45 port 51237 ssh2
Sep 07 14:20:28 anshul-kali sshd[2105]: Failed password for invalid user admin from 10.0.2.45 port 51238 ssh2
Sep 07 14:21:01 anshul-kali sshd[2107]: Failed password for anshul from 10.0.2.18 port 51239 ssh2
Sep 07 14:21:06 anshul-kali sshd[2108]: Failed password for anshul from 10.0.2.18 port 51240 ssh2
Sep 07 14:23:02 anshul-kali sshd[2112]: Failed password for invalid user guest from 10.0.2.45 port 51242 ssh2
```

Purpose: Extracts only the log rows corresponding to authentication failures.

2. Quantifying Failures:

```
grep -c "Failed password" auth_security.log
```

```
┌──(kali㉿kali)-[~/log]
└─$ grep -c "Failed password" auth_security.log
8
```

Purpose: Returns an exact integer count of total failed password entries.

3. Extracting and Tallying Source IPs:

```
grep "Failed password" auth_security.log | awk '{for(i=1;i<=NF;i++) if($i=="from") {print $(i+1); break}}' | sort | uniq -c
```

```
┌──(kali㉿kali)-[~/log]
└─$ grep "Failed password" auth_security.log | awk '{for(i=1;i<=NF;i++) if($i=="from") {print $(i+1); break}}' | sort | uniq -c
      2 10.0.2.18
      6 10.0.2.45
```

Purpose: Isolates failure events, dynamically extracts the IP address immediately following the keyword 'from', sorts them alphabetically, and counts occurrences per IP.

5.2 Text Normalization with sed

```
sed 's/[[:space:]]\+/ /g' auth_security.log
```

```
┌──(kali㉿kali)-[~/log]
└─$ sed 's/[[:space:]]\+/ /g' auth_security.log
Sep 07 14:20:11 anshul-kali sshd[2101]: Failed password for invalid user admin from 10.0.2.45 port 51234 ssh2
Sep 07 14:20:15 anshul-kali sshd[2102]: Failed password for invalid user root from 10.0.2.45 port 51235 ssh2
Sep 07 14:20:19 anshul-kali sshd[2103]: Failed password for invalid user test from 10.0.2.45 port 51236 ssh2
Sep 07 14:20:23 anshul-kali sshd[2104]: Failed password for invalid user guest from 10.0.2.45 port 51237 ssh2
Sep 07 14:20:28 anshul-kali sshd[2105]: Failed password for invalid user admin from 10.0.2.45 port 51238 ssh2
Sep 07 14:21:01 anshul-kali sshd[2107]: Failed password for anshul from 10.0.2.18 port 51239 ssh2
Sep 07 14:21:06 anshul-kali sshd[2108]: Failed password for anshul from 10.0.2.18 port 51240 ssh2
Sep 07 14:22:15 anshul-kali sshd[2110]: Accepted password for anshul from 10.0.2.5 port 51241 ssh2
Sep 07 14:23:02 anshul-kali sshd[2112]: Failed password for invalid user guest from 10.0.2.45 port 51242 ssh2
```

Purpose: Cleans up irregularities by replacing consecutive whitespaces with a single uniform space, standardizing log entries for downstream text processing.

5.3 Event Extraction with awk

```
awk '/Accepted password/ {print "Successful login:", $0}' auth_security.log
```

```
┌──(kali㉿kali)-[~/log]
└─$ awk '/Accepted password/ {print "Successful login:", $0}' auth_security.log
Successful login: Sep 07 14:22:15 anshul-kali sshd[2110]: Accepted password for anshul from 10.0.2.5 port 51241 ssh2
```

Purpose: Scans the log stream for successful authentications and formats the output for immediate visibility.

6. Automation Bash Script for Log Analysis

To streamline repetitive log triage, the following modular Bash script (threat_detect.sh) was written to automate parsing, data cleaning, and anomaly reporting.

```
#!/bin/bash

# Configuration Variables
LOG_FILE="auth_security.log"
CLEANED_LOG="cleaned_security.log"
THRESHOLD=3

# Validate Log File Existence
if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Log file '$LOG_FILE' not found."
    exit 1
fi

echo "=== SSH SECURITY LOG ANALYSIS ==="
echo

# Normalize whitespace
sed 's/[[:space:]]\+/ /g' "$LOG_FILE" > "$CLEANED_LOG"

# Count totals
FAILED=$(grep -c "Failed password" "$CLEANED_LOG")
SUCCESS=$(grep -c "Accepted password" "$CLEANED_LOG")

echo "Total failed login attempts: $FAILED"
echo "Total successful login attempts: $SUCCESS"
echo

echo "Failed login attempts grouped by IP address:"
grep "Failed password" "$CLEANED_LOG" | \
    awk '{for(i=1;i<=NF;i++) if($i=="from") {print $(i+1); break}}' | \
    sort | uniq -c
echo

echo "Suspicious IP addresses (Exceeding threshold of $THRESHOLD failures):"
grep "Failed password" "$CLEANED_LOG" | \
    awk '{for(i=1;i<=NF;i++) if($i=="from") {print $(i+1); break}}' | \
    sort | uniq -c | \
    awk -v limit="$THRESHOLD" '$1 >= limit {print $2, "-", $1, "failed attempts"}'
```

```
echo
echo "=== ANALYSIS COMPLETE ==="
```

7. Execution Procedure

Execute the analysis script on your Kali Linux terminal via the following sequence:

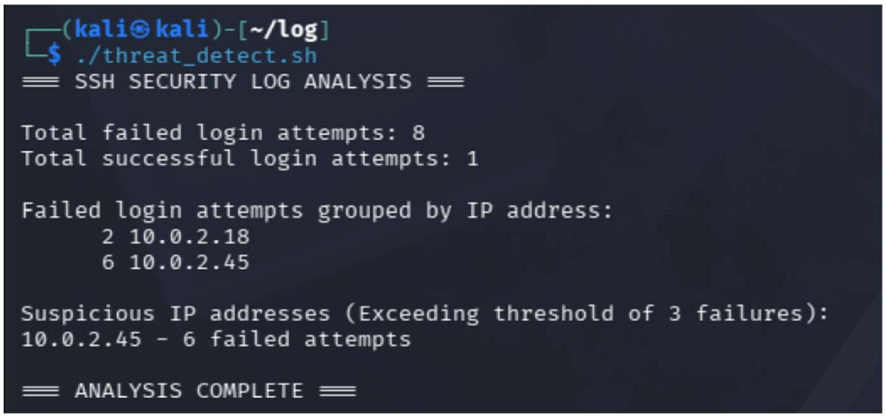
- **Grant execution permissions:**

```
chmod +x threat_detect.sh
```



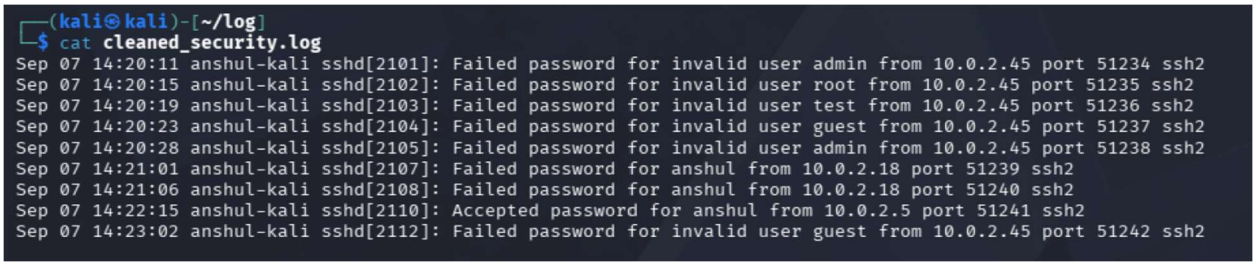
- **Run the script:**

```
./threat_detect.sh
```



- **Verify generated output reports:**

```
cat cleaned_security.log
```



8. Output and Identification of Suspicious Activities

Running the automated script against the sample dataset yields the following structured output:

```
=== SSH SECURITY LOG ANALYSIS ===

Total failed login attempts: 8
Total successful login attempts: 1

Failed login attempts grouped by IP address:
      2 10.0.2.18
      6 10.0.2.45

Suspicious IP addresses (Exceeding threshold of 3 failures):
10.0.2.45 - 6 failed attempts

=== ANALYSIS COMPLETE ===
```

Key Analytical Findings:

- A total of **8 failed authentication attempts** were logged.
- Exactly **1 successful login** was registered during the session window.
- Source IP **10.0.2.45** accounted for 6 distinct failures targeting multiple invalid user accounts.
- Source IP **10.0.2.18** generated 2 failures.
- IP **10.0.2.45** successfully breached the defined security threshold and was flagged for intensive follow-up.

9. Justification of the Security Threshold

A threshold limit of three or more failed attempts was established for this assessment due to the following rationale:

- **Pattern Recognition:** Rapid consecutive failures strongly correlate with automated credential stuffing or brute-force scripts.
- **Signal-to-Noise Balance:** Helps isolate malicious scanning activity from occasional human typing errors.
- **Simplicity and Adaptability:** Easy to code and scale, serving as an effective baseline for lightweight log analysis scripts.

10. False Positives and Limitations

10.1 Potential False Positives

- **User Error:** Legitimate system users might mistype complex passwords multiple times.
- **Shared Network Gateways:** Multiple corporate or academic users behind a single NAT/public IP address can aggregate isolated failures.
- **Stale Credentials:** Background cron jobs, lingering API keys, or automated scripts configured with outdated passwords can trigger continuous faults.

10.2 Technical Limitations

- The analysis relies on a static, simulated log snippet rather than real-time system monitoring streams.
- Basic failure counting does not factor in precise time deltas (velocity/frequency of requests over time).
- Advanced adversaries utilizing distributed botnets across dozens of unique IP addresses can stay safely beneath individual volume thresholds.